Discrete Optimization

Kulczycki Robin, Paulis Antoine and Zanella Francois

Academic year 2025-2026

Project report

EURO/ROADEF 2026 Challenge - *T-Adaptive Segment Routing problem*

LIÈGE université
Sciences Appliquées

1 MIP model

In order to model the problem in a MIP model, let us first consider the following expressions of the variables, constraints and objective function of T-ASR.

1. **Variables** :

   (a) **The binary decision variables** :

$$x_{i,j}^{dt} = \begin{cases} 1 & \text{if the segment path of demand } d \in D \text{ contains } (i,j) \text{ at time step } t \in T \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

   (b) **The load variables** : The load $\lambda(a,t) \geq 0$ is the ratio between the quantity of flow using an arc and its capacity . Note that it can be bigger than 1 in the case of congestion on the link. It can be expressed as follows

$$\lambda(a,t) = \frac{\sum_{d \in D} \sum_{i,j \in V} r(i,j,a,t)\, \nu(d,t)\, x_{ij}^{d,t}}{c(a)} \tag{2}$$

2. **Constraints** :

   (a) <u>Flow conservation constraints</u> :

$$\sum_{j \in V \setminus \{i\}} x_{ij}^{d,t} - \sum_{j \in V \setminus \{i\}} x_{ji}^{d,t} = \begin{cases} 1 & \text{if } i = s, \\ -1 & \text{if } i = t, \\ 0 & \text{otherwise.} \end{cases} \qquad \forall i \in V,\ \forall d = (s,t) \in D,\ \forall t \in T \tag{3}$$

   (b) <u>Number of segments</u> :

$$\sum_{i,j \in V} x_{ij}^{d,t} \leq \text{maxSeg}, \quad \forall d \in D,\ \forall t \in T \tag{4}$$

   (c) <u>Load constraints</u> :

$$\sum_{d \in D} \sum_{i,j \in V} r(i,j,a,t)\, \nu(d,t)\, x_{ij}^{d,t} \leq \lambda(a,t)\, c(a), \quad \forall t \in T,\ \forall a \in A \setminus q(t) \tag{5}$$

   (d) <u>Budget constraints</u> :

$$\sum_{d \in D} \sum_{i,j \in V,\ i \neq j} \left| x_{ij}^{d,t} - x_{ij}^{d,t-1} \right| \leq \kappa(t), \quad \forall t \in T^* \tag{6}$$

   To linearize this constraint, we can introduce a variable $y_{ij}^{dt} \geq 0$ with the following behaviour :

$$\sum_{d \in D} \sum_{i,j \in V,\ i \neq j} y_{ij}^{dt} \leq \kappa(t) \qquad \forall t \in T^* \tag{7}$$

$$y_{ij}^{dt} \geq x_{ij}^{dt} - x_{ij}^{d,t-1} \qquad \forall d \in D,\ \forall i,j \in V,\ i \neq j,\ \forall t \in T^* \tag{8}$$

$$y_{ij}^{dt} \geq -x_{ij}^{dt} + x_{ij}^{d,t-1} \qquad \forall d \in D,\ \forall i,j \in V,\ i \neq j,\ \forall t \in T^* \tag{9}$$

   (e) <u>Subtour Elimination constraint</u> :

$$\sum_{i \in V : (i,v) \in S_t} x_{iv}^{d,t} \leq 1, \quad \forall d \in D\ ,\ \forall t \in T \text{ and } \forall v \in V \setminus \{s_d, t_d\},$$

   where S_t is the set of deliverable segments at time t. This constraint ensures that at most one segment can enter v in the routing of demand d at time t.

3. **Objective** : Let $L = \{\lambda(a,t) \mid a \in A,\ t \in T\}$ be the set of loads of all arcs at all time steps.

$$\text{lex min } \{\lambda'(i) : 1 \leq i \leq |L|\} \tag{10}$$

Note that the way we will express this objective will be handled in the next section.

4. **Offline computation** : The split coefficients $r(i,j,a,t)$ represent the proportion of flow between a source node u and a target node v that flows through arc a and they are used in the load constraints (5). They can be computed offline since they only depend on the network topology, arc weights and failures but not on the decision variables. The algorithm to compute the split coefficients is described as follows :

   (a) For each time step t, the algorithm computes the graph G_t computed by removing all the unavailable arcs due to the interventions.

   (b) The algorithm runs *Dijkstra* thanks to the*networkx* library, to get all the shortest-path distances for every possible segment (i,j) and also the reverse graph that provides the shortest-path distances toward any target.

   (c) The algorithm builds the forwarding graph $FG(i,j)$ for each pair of nodes (i,j). To do so, it checks for each arc (u,v) if it belongs to any shortest-path by computing the $dist^*(i,j)$ (denoting the shortest-path distance from i to j) and computing $dist^*(i,u)+\omega(u,v)+dist^*(u,v)$ (denoting the shortest-path distance from i to j, passing through the arc (u,v)). Then, it will check if $dist^*(i,j) = dist^*(i,u) + \omega(u,v) + dist^*(v,j)$ which means that the arc (u,v) belongs to any shortest-path, and then, (u,v) belongs to $FG(i,j)$.

   (d) The algorithm applies the *Equal-Cost Multi-Path* (ECMP) mechanism which means that the incoming flow in a vertex of $FG(u,v)$ is evenly divided between the outgoing arcs of this vertex in $FG(u,v)$. First, for each pair of nodes (i,j), we will initialize the flow f such that $f(i) = 1$ and $f(v) = 0, \quad u \neq i$. Then, it processes the nodes v that belong to $FG(i,j)$, in an increasing order based on the $dist^*(u,j)$. Then, for each node u, it splits the flow equally such that for each successor v of u, we have :

$$f(v) = f(v) + \frac{f(u)}{|successor(u)|} \qquad \text{and} \qquad r(i,j,u,v,t) = r(i,j,u,v,t) + \frac{f(u)}{|successor(u)|}.$$

Then, in order to speed up the computation of the split coefficients $r(i,j,a,t)$, we have implemented a faster method. It follows the same structure as the previous one but instead of using the *networkx* library, this method follows those guidelines :

1. Uses a custom *Dijkstra* which returns a distance matrix per time slot.

2. Instead of recomputing *Dijkstra* on the reverse graph, considers the transpose of the matrix returned by the custom *Dijkstra*.

2 Algorithm

Firstly, let's investiguate some methods in order to handle the objective. From Wikipedia (`https://en.wikipedia.org/wiki/Lexicographic_optimization`), we have found a way to express the minimization objective as a lexicographic order. Those L objectives can be solved by tackling n single-objective problems such as
For $l = 1$ to $|L|$ **do:**

$$\begin{aligned}
\min \quad & z && (11)\\
\text{s.t.} \quad & \text{(a)-(d)} && (12)\\
& \lambda(a,t) \leq z, \quad \forall a \in A, \forall t \in T && (13)\\
& \lambda(a,t) \leq z^k, \forall k \in \{1, ..., l-1\} && (14)
\end{aligned}$$

If the problem is unbounded or infeasible then, no solution.
Otherwise, set $z_l = z^*$
End for
So those expressions respect the lexicographic minimization objective. Since (11) minimizes z and (13) sets z to the maximum value of the c. This will lead to the minimization of the maximum value of the $\lambda(a,t)$. Moreover, (14) ensures that for each previous iteration, all $\lambda(a,t)$ are bounded by the previous optimal z^k value found in those previous iterations. This will allow that the solver find a new minimum z which is smaller than the previous one and it will lead to $z^1 \geq z^2 \geq ... \geq z^l$.at each iteration we identify the binding arcs of the current optimum (those with (a,t) = z*) and freeze only those by replacing their cap (a,t) z with (a,t) z*. Non-binding arcs keep the cap (a,t) z and remain free to decrease in subsequent iterations.

3 Heuristic and warm start

In order to accelerate the MIP solve, we couple it with a constructive heuristic that we use to warm-start Gurobi. The general idea is the following:

1. We first compute a feasible solution with a greedy heuristic, called H1.
2. Then, we encode this solution into the MIP variables x_{ij}^{dt}, y_{ij}^{dt}, $\lambda(a,t)$ and z, and pass them to Gurobi through the `Start` attribute, so that Gurobi treats the heuristic's solution as its initial incumbent.
3. Finally, Gurobi runs its branch-and-bound and tries to improve this incumbent within the per-instance time limit.

The heuristic H1 is a greedy demand-insertion procedure. Firstly, the demands $d \in D$ are sorted in decreasing order of their total volume $\sum_{t \in T} \nu(d,t)$, since the bigger demands are the most likely to create congestion if routed at the wrong place. Then, each demand is inserted one by one in the time slots $t \in T$, on the candidate path that least increases the maximum link utilization (MLU) at this slot. The candidate paths considered for a demand $d = (s,t)$ are the direct segment (s,t) and the 1-waypoint paths $(s,w) + (w,t)$ for each $w \in V \setminus \{s,t\}$, which corresponds to the case maxSeg ≤ 2. The reconfiguration budget $\kappa(t)$ is also checked at each insertion, so that the heuristic always returns a solution feasible w.r.t. constraint (6).

there are two reasons twouse H1 as a warm start for gurobi:

1. The MLU returned by H1 immediately becomes an upper bound on the objective z. therefore, the branch-and-bound tree is heavily pruned: any node whose LP relaxation already exceeds this bound is cut, which typically removes large parts of the search space even before the root LP is solved.
2. The solver is guaranteed to never return a solution worse than H1. Indeed, even if the 10-minute per-instance cap fires before Gurobi has proved optimality, the incumbent returned by Gurobi is by construction at least as good as the heuristic's solution.

Finally, when combined with the lexicographic loop described in equations (11)–(14), the warm start mechanism is even more interesting. At each iteration l of the lex loop, the solution obtained at iteration $l-1$ is feasible for iteration l (since iteration l only adds the constraint $\lambda(a,t) \leq z^{l-1}$ on the binding arcs). We then reuse it as the warm start of iteration l, so that the search effort accumulated during the previous iterations is preserved instead of being restarted from scratch.

4 Column restriction methods

4.1 Motivation

The MIP formulation contains a binary variable $x_{ij}^{d,t}$ for every demand $d \in D$, time slot $t \in T$, and deliverable segment $(i,j) \in S_t$. The total number of variables is therefore $|D| \times |S_t| \times |T|$, which grows very rapidly with instance size. For large instances (e.g for instance 13, we have $|D| = 2,000$ and $|S_t| = 39,800$

with $|T| = 2$), this produces nearly 160 million variables, making model construction alone infeasible within the 10-minute time cap. The idea of column restriction is to replace the full deliverable set S_t with a much smaller per-demand candidate set $C_{d,t} \subseteq S_t$, such that $|C_{d,t}| \leq K$ for a parameter $K \ll |S_t|$. The MIP is then solved over this restricted variable set, reducing model size from $|D| \times |S_t|$ to $|D| \times K$ variables. The quality of the restriction depends entirely on how $C_{d,t}$ is constructed, ideally it should contain the segments that appear in the true optimal solution. Two methods are proposed depending on the instance size.

4.2 MLU scoring

For each demand d and each candidate segment $(i, j) \in S_t$, a score is computed that estimates the load impact of routing demand d through segment (i, j):

$$score(d, t, i, j) = \max_{a \in A} \frac{f_{H1}(a, t) + r(i, j, a, t) v_{d,t}}{c_a}$$

where $f_{H1}(a, t)$ is the flow on arc a at time t in the H1 solution, $r(i, j, a, t)$ is the ECMP split coefficient (fraction of flow on segment (i, j) passing through arc a), $v_{d,t}$ is the volume of demand d at time t, and c_a is the capacity of arc a. This score represents the MLU that would result if demand d were routed through segment (i, j), on the top of the existing H1 flow[1]. A low score means the segment routes demand d without heavily loading any arc, making it a good candidate. The K segments with the lowest scores are kept in $C_{d,t}$.
Furthermore, two additional guarantees are enforced :

1. The direct path (s_d, t_d) is always included if deliverable, since it represents the simplest valid routing.
2. All segments used by the H1 warm start are always included, which guarantees that the MIP can always reproduce the H1 solution and never returns a worse result.

The complexity of this method is $\mathbf{O}(|D| \times |S_t| \times a_{avg})$ where a_{avg} is the average number of arcs per segment.

4.3 Reachability filter

For very large instances where the scoring method becomes prohibitive, a purely topological filter is used instead. The key observation is that for a demand d going from s_d to t_d with at most `maxSeg` segments, a segment (i, j) can only appear in a valid path if the two following conditions are simultaneously satisfied :

- Node i is reachable from s_d in at most `maxSeg` $- 1$ hops through deliverable segments
- Node j can reach t_d in at most `maxSeg` $- 1$ hops through deliverable segments

Segments that fail one of the two conditions cannot appear in any valid solution for demand d and can be safely discarded. The reachability sets

$$\text{forward}(v) = \{\omega \in V : \omega \text{ reachable from } v \text{ in } 1, ..., \texttt{maxSeg} - 1 \text{ hops}\}$$
$$\text{backward}(v) = \{\omega \in V : v \text{ reachable from } \omega \text{ in } 1, ..., \texttt{maxSeg} - 1 \text{ hops}\}$$

are precomputed once per time slot using *Breadth-First Search* procedure

```
frontier = {source}
visited = {}
for k = 1 to maxSeg-1:
    new_frontier = all neighbors of nodes in frontier
    frontier = new_frontier - visited
    visited = visited + frontier
```

Figure 1: Breadth-First Search procedure

[1]It is an additive approximation that overestimates the true load impact (since the original H1 flow is not removed) but this is acceptable because we only need a ranking of segments, not exact load values.

such that after k iterations, `visited` contains all nodes reachable in exactly 1 to k hops from the source (forward BFS). The same idea is applied to find the nodes that can reach a target node (backward BFS). The candidate set for demand d at time t is then

$$C_{d,t} = \{(i,j) \in S_t : i \in \text{forward}(s_d) \cup \{s_d\} \text{ and } j \in \text{backward}(t_d) \ \cup \ \{t_d\}\}$$

truncated to the K first elements. The direct path and H1 warm start segments are again always included. It is important to note that in this case, the parameter K acts as a purely arbitrary truncation.

4.4 Method selection

The method is selected automatically based on the estimated number of scoring operations given by $n_{ops} = |D| \times |S_t| \times |T|$ such that

Condition	**K**	**Method**
$n_{ops} \leq 50,000$	*none*	No restriction
$50,000 < n_{ops} \leq 100 \times 10^6$	50	MLU Scoring
$n_{ops} > 100 \times 10^6$	20	Reachability filter

Table 1: Thresholds for column restriction method selection

In practice, this corresponds to the following configuration :

Instances	**Method**
01-03	none
04-12	scoring($K = 50$)
13	reachability($K = 20$)
14-15	scoring ($K = 50$)
16-20	reachability($K = 20$)

Table 2: Thresholds for column restriction method selection

4.5 Trade-off

The parameter K directly controls the trade-off between model size and solution quality. A larger K gives more routing alternatives and leads to better results but increases model construction time. A smaller K however decreases solving time but restricts the search space. On the test set, $K = 50$ was used for the scoring method and $K = 20$ for the reachability method. These values were retained as they provided acceptable results in practice.

5 Results and performance analysis

Before starting the analysis, two important comments should be made. First, we did not manage to obtain better solutions than H1 for instances 19 and 20 of set A, because the construction time was so long that it could not be achieved within the 10-minute time cap. These two instances are hence not considered into the analysis. Furthermore, the lexicographic minimization was purposely limited to the first 5 ranks of the sorted load vector. Knowing that, let us know analyse the results.

Firstly looking at our MLU results for H1 and the MIP we can see that our heuristic almost always gave the right MLU which means it was good from the start and therefore prune a significant amount of values.

Inst.	Size			H1		MIP			Status
	$\|V\|$	$\|A\|$	$\|D\|$	MLU	Cost	MLU	Cost		
01	20	80	40	0.9294	49	0.9294	47	282.9200	opt
02	30	150	45	0.9031	27	0.9031	46	69.7700	opt
03	50	250	20	0.9435	18	0.9435	47	104.6400	opt
04	50	250	200	0.7138	42	0.7138	21	5.0600	opt
05	100	396	100	0.2050	0	0.2050	0	8.8600	opt
06	100	500	500	0.1408	11	0.1408	4	43.0200	opt
07	100	500	800	0.9080	90	0.9080	33	57.6100	opt
08	150	654	200	0.3189	13	0.3189	8	77.8800	opt
09	150	750	200	0.8497	18	0.8497	2	44.4700	opt
10	150	966	1000	0.1022	0	0.0978	0	347.0300	opt
11	200	1000	400	0.7858	21	0.7858	4	242.2200	opt
12	200	898	400	0.8799	12	0.8799	12	285.6800	opt
13	200	1000	2000	0.0491	12	0.0491	6	181.4100	opt
14	250	1108	600	0.8445	11	0.5721	0	440.9200	opt
15	250	1250	600	0.8987	12	0.8987	4	318.2200	opt
16	250	1452	4800	0.1906	11	0.1906	11	600.0000	H1 only
17	300	1270	2000	0.4242	0	0.4242	0	530.1600	opt
18	300	1500	2000	1.0000	57	1.0000	0	440.8000	opt

Table 3: Per-instance results

The next figure shows a comparison between our H1 solution and our MIP solution. We can see the dashed-line showing about to where our loads were fully optimized (lex_iter = 5). We can also see that our MIP stays better than our greedy heuristic for all values up to 50. It is because ranks 6-50 inherit benefits from the optimisation of ranks 1-5. meaning the usage of the MIP of sheerly H1 was significant.

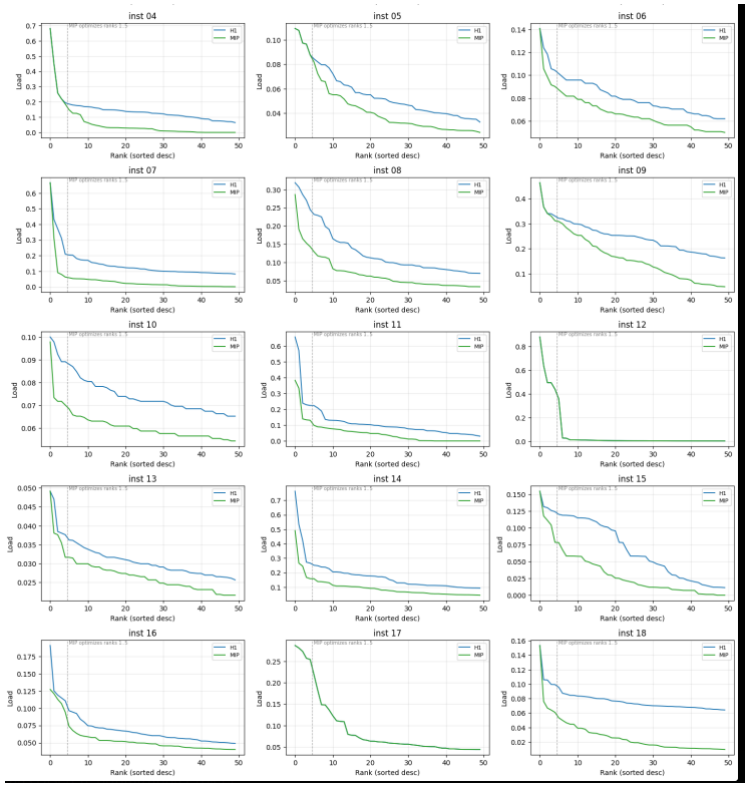

Figure 2: H1 vs MIP on first 50 lex-values

For the next table showing the best values for the first 5 optimized values, we can see that for most instances there was highly congested link then a significant drop. A second group (instances 9, 15, 17) shows a more uniform decline across the five ranks, characteristic of networks where multiple arcs are simultaneously close to the MLU and the lex loop must reduce them one by one rather than removing a single dominant bottleneck.

Inst.	z_1^*	z_2^*	z_3^*	z_4^*	z_5^*
01	0.9294	0.5520	0.5078	0.4898	0.4757
02	0.9031	0.5486	0.5388	0.5228	0.4955
03	0.9435	0.6216	0.5898	0.5853	0.5730
04	0.7138	0.6803	0.5613	0.4623	0.4504
05	0.2050	0.1155	0.1092	0.1085	0.1076
06	0.1408	0.1056	0.1014	0.0986	0.0958
07	0.9080	0.6656	0.6652	0.6643	0.6616
08	0.3189	0.3069	0.2864	0.2359	0.2319
09	0.8497	0.8069	0.7415	0.5883	0.5551
10	0.0978	0.0870	0.0783	0.0761	0.0739
11	0.7858	0.6167	0.6119	0.6100	0.5011
12	0.8799	0.6401	0.4949	0.4400	0.3600
13	0.0491	0.0470	0.0402	0.0389	0.0385
14	0.5721	0.4937	0.4549	0.4397	0.4101
15	0.8987	0.8847	0.8425	0.8252	0.8150
17	0.4242	0.4100	0.3655	0.3457	0.3132
18	1.0000	0.7131	0.6334	0.6148	0.4971

Table 4: Optimal load z_k^* at lex rank k for $k = 1, \ldots, 5$. Rank 1 is the MLU. Values are monotone non-increasing by construction of the lex loop. Instance 16 is omitted (no MIP solution).

6 Strengths and limitations

6.1 Heuristic

The heuristic H1 is a fast and effective greedy algorithm that provides valid and competitive solutions in a fraction of seconds even on the largest instances. However, it is currently used solely to provide a warm start for the MIP, meaning its contribution is limited to a single feasible solution injected at the beginning of the solving process. Since H1 runs in seconds even on the largest instances, it could be used more extensively. For example by using it multiple times with different demand orderings and keeping the best results. This would be particularly valuable for instances 19 and 20 where MIP fails entirely, leaving H1 as the only source of solution with no further improvement.

6.2 MLU scoring

MLU scoring method significantly reduces the number of variables, thereby considerably decreasing the time required to build the model. However, there are two main limitations of it.

1. It measures the impact of a segment on the most loaded arc, which is exactly what rank-0 (MLU) cares about. Hence, segments that are good for rank-0 get low scores and are kept. However, segments that are irrelevant for rank-0 but would help ranks 1-5 get high scores and are discarded. So the column restriction is built using only rank-0 information. So by the time the MIP tries to optimize rank-1, rank-2, ..., the segments that would allow better solutions at those ranks have already been eliminated from the candidate set. A possible improvement would be to score segments based on their impact of the full sorted load vector, not just the maximum. For example, using the sum of loads as a secondary criterion. It would retain segments that are neutral on rank-0 but beneficial for lower ranks. However, it increases scoring complexity.

2. Another limitation is that K is a global parameter, which means the same value is applied to all demands, regardless of their individual characteristics. However, some demands may require totally different numbers of candidates K, depending on the connectivity between the source and the target nodes of the demand. For example, $K = 50$ candidates for a demand between two highly connected nodes could be too small, but for two poorly connected nodes it could be too large.

7 AI declarations

The authors have used generative AI to help the programming task. However, everything that was generated has been understood. The authors take full accountability for the results.